

# AR Drone Concept (Pre-Flight Hand Dodging Simulation)

Documentation by: Krishay Chaddha

## Objective:

Before buying some of the physical parts and hardware for the drone, I wanted to create a safe and reliable way to test out hand-tracking and obstacle dodging mechanics. I built this simulation entirely by myself (with a little help from AI for brainstorming ideas) using Python, Pygame, and OpenCV to see how a hand could dodge certain obstacles in a 3D space.

## Step by Step Process:

Step 1: In order to simulate a 3D environment on a 2D screen, I need ways for the objects on the screen to approach the user rather than moving left and right. This will be helpful as it will be closer to how a drone would approach objects in the real world.

- Z-Axis Progression: To create the 3D environment, I created a Pipe class where obstacles would start far in the background ( $z=0.2$ ) and steadily increase their depth value over time (in a randomized approach speed)

Step 2: Next, I needed a manager to control how obstacles appear and update so that the simulation could run continuously without crashing or lagging.

- Timed Spawning: I added a spawn\_timer which automatically generates new obstacles at intervals
- Survival and Scoring: I also built logic to automatically reward the player a point if an obstacle safely passes the threshold without a collision. This is necessary for a drone AR racing game because if it is trained using an ML model, the AI would recognize that additional points are necessary.

Step 3: I integrated OpenCV next for live frame rendering.

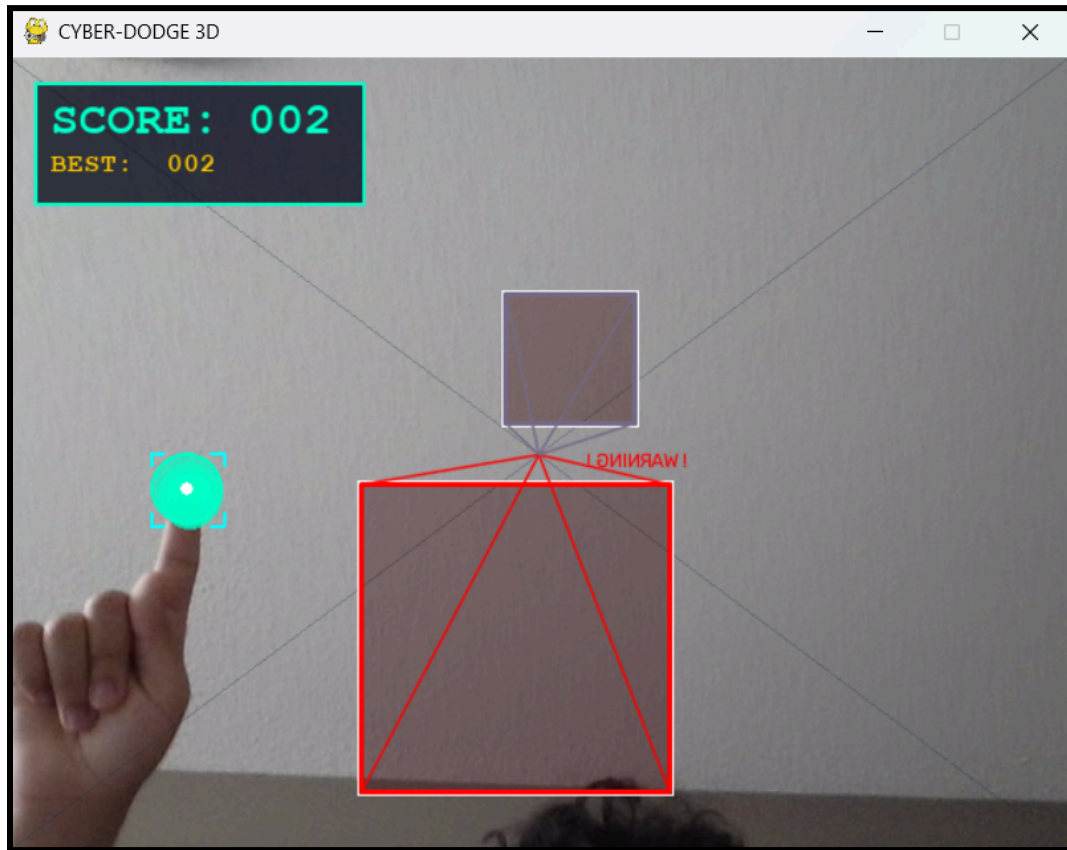
- For example, I programmed a dynamic colour calc that shifts the obstacle's wireframe and polygon colors from cool cyan/blue to red as it gets closer. This is also important for a drone as the camera it will utilize will encompass a colour sensor which will be an additional way to handle obstacle proximity.
- I also layered text prompts such as (! Warning !) that triggers when an obstacle enters the front-view zone.

Step 4: I created a collision detection system to test whether my hand dodged the obstacle or not (similar to how a drone would do it)

- I created collision checks (check\_front\_face\_collisions) to catch direct hits right when objects enter the danger zone, deducting points. I also inflated the Pygame tracking rectangles slightly to make near-misses feel more responsive.

This simulation is a great resource for me when I actually get to create the final drone project and utilize the same algorithms to train a drone to autonomously navigate and dodge moving obstacles using real time camera feeds.

Image:



**Figure 1: A picture of the Game with real-time obstacles and Warning Symbols. A feed with a real-time score that is either deducted from or added to.**

Video: [https://drive.google.com/file/d/1WpUZ66Vgvz\\_oJuywh\\_WhcU2FPDHLnUlt/view?usp=sharing](https://drive.google.com/file/d/1WpUZ66Vgvz_oJuywh_WhcU2FPDHLnUlt/view?usp=sharing)